

final project python development_PITP Batch-3 submiteed by: Iqra Trainer: Engr. Uzaif talpur

```
"""
Pakistan Job Market Analysis & Forecasting
Dec 2019 – Mar 2021
Complete Data Science Project: EDA, Cleaning, ML Modeling,
Visualization, PDF Report
"""

#


---


# INSTALL REQUIRED PACKAGES (run this once in terminal if not
installed):
# pip install pandas numpy matplotlib seaborn scikit-learn reportlab
#


---


# IMPORTS
import pandas as pd
import numpy as np
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
import matplotlib.gridspec as gridspec
from matplotlib.colors import LinearSegmentedColormap
from matplotlib.ticker import MaxNLocator
import seaborn as sns
import warnings, os, re, textwrap
from collections import Counter
from datetime import datetime

# ML imports
from sklearn.ensemble import RandomForestClassifier,
GradientBoostingRegressor
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.metrics import classification_report, mean_squared_error,
r2_score
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA

# PDF
from reportlab.lib.pagesizes import A4
from reportlab.lib import colors
from reportlab.lib.units import inch, cm
```

```

from reportlab.lib.styles import getSampleStyleSheet, ParagraphStyle
from reportlab.lib.enums import TA_CENTER, TA_LEFT, TA_JUSTIFY
from reportlab.platypus import (SimpleDocTemplate, Paragraph, Spacer,
                                Image as RImage, Table, TableStyle,
                                HRFFlowable, PageBreak, KeepTogether)
from reportlab.lib.colors import HexColor

warnings.filterwarnings('ignore')
sns.set_theme(style="whitegrid")

#


---


# □ FILE PATHS – UPDATE THESE IF NEEDED
#


---


CSV_PATH = r"C:\Users\asus\Pakistan Available Job Dec 19 - Mar-
21.csv"
OUT_DIR = r"C:\Users\asus\JobMarketAnalysis" # outputs
folder
FIG_DIR = r"C:\Users\asus\JobMarketAnalysis\figures" # chart images

os.makedirs(OUT_DIR, exist_ok=True)
os.makedirs(FIG_DIR, exist_ok=True)

#


---


# COLOUR PALETTE
#


---


PALETTE = {
    "green": "#2ECC71",
    "blue": "#2980B9",
    "orange": "#E67E22",
    "red": "#E74C3C",
    "purple": "#8E44AD",
    "teal": "#16A085",
    "yellow": "#F1C40F",
    "navy": "#2C3E50",
    "pink": "#EC407A",
    "indigo": "#5C6BC0",
    "cyan": "#00BCD4",
    "brown": "#795548",
}
COLORS = list(PALETTE.values())
CMAP = LinearSegmentedColormap.from_list("pkjobs",
[ "#2980B9", "#2ECC71", "#E67E22", "#E74C3C" ])

```

```
def save_fig(name):
    path = os.path.join(FIG_DIR, f"{name}.png")
    plt.savefig(path, dpi=150, bbox_inches='tight', facecolor='white')
    plt.close()
    print(f" ✓ Saved figure: {name}.png")
    return path
```

```
#
```

```
# 1. LOAD & CLEAN DATA
```

```
#
```

```
print("\n" + "="*60)
print(" STEP 1: Loading & Cleaning Data")
print("="*60)
```

```
df = pd.read_csv(CSV_PATH, on_bad_lines='skip')
```

```
print(f" Raw shape      : {df.shape}")
print(f" Columns       : {df.columns.tolist()}")
print(f" Missing values:\n{df.isnull().sum()}")
```

```
# Column cleanup
```

```
df.columns = df.columns.str.strip()
```

```
# Parse dates
```

```
df['Date Posted'] = pd.to_datetime(df['Date Posted'], dayfirst=True,
errors='coerce')
df.dropna(subset=['Date Posted'], inplace=True)
df['Year'] = df['Date Posted'].dt.year
df['Month'] = df['Date Posted'].dt.month
df['YearMonth'] = df['Date Posted'].dt.to_period('M')
df['MonthName'] = df['Date Posted'].dt.strftime('%b %Y')
df['DayOfWeek'] = df['Date Posted'].dt.day_name()
```

```
# Clean text columns
```

```
for col in ['Job Name', 'Department', 'City', 'Experience Required', 'Job
Type', 'label']:
    if col in df.columns:
        df[col] = df[col].fillna('Unknown').astype(str).str.strip()
```

```
# Normalize Department
```

```
df['Dept_Clean'] = (df['Department']
                    .str.replace(r'\s+Jobs?$', '', regex=True,
flags=re.IGNORECASE)
                    .str.replace(r'\s+Job$', '', regex=True,
flags=re.IGNORECASE)
```

```

        .str.strip())

# Normalize Experience
exp_map = {
    'Job for Fresh Graduates': 'Fresher',
    '< 1 Year': '< 1 Year',
    '1 Year Job Exp.': '1 Year',
    '2 Years Job Exp.': '2 Years',
    '3 Years Job Exp.': '3 Years',
    '4 Years Job Exp.': '4 Years',
    '5 Years Job Exp.': '5 Years',
    '6 Years Job Exp.': '6+ Years',
    '7 Years Job Exp.': '6+ Years',
}
df['Exp_Clean'] = df['Experience
Required'].map(exp_map).fillna('Other')
df['JobType_Clean'] = df['Job Type'].str.replace(r'\s+Jobs?$', '',
regex=True, flags=re.IGNORECASE).str.strip()
df['Is_Online'] = df['Job Name'].str.contains('online',
case=False, na=False)
df['Is_Remote'] = (df['Job Name'].str.contains('remote',
case=False, na=False) |
df['JD'].fillna('').str.contains('remote',
case=False, na=False))

# Extract tech skills from JD
SKILLS =
['Python', 'Java', 'PHP', 'JavaScript', 'React', 'Angular', 'Node', 'SQL',

'WordPress', 'Flutter', 'iOS', 'Android', '.NET', 'C#', 'Laravel', 'Django',
'Machine Learning', 'AI', 'Data
Science', 'Excel', 'SAP', 'AutoCAD']

for sk in SKILLS:
    df[f'skill_{sk}'] = df['JD'].fillna('').str.contains(sk,
case=False, na=False).astype(int)

print(f"\n Cleaned shape : {df.shape}")
print(" ✓ Data cleaning complete.")

#


---


# 2. COMPUTED METRICS
#


---


print("\n" + "="*60)
print(" STEP 2: Computing Metrics")
print("="*60)

```

```

monthly_counts =
df.groupby('YearMonth').size().reset_index(name='Count')
monthly_counts['YearMonth_str'] =
monthly_counts['YearMonth'].astype(str)

dept_counts = df['Dept_Clean'].value_counts().head(15)
city_counts = df['City'].value_counts().head(10)
exp_counts = df['Exp_Clean'].value_counts()
label_counts = df['label'].value_counts()
dow_counts = df['DayOfWeek'].value_counts().reindex(
['Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday', 'Saturday', 'Sunday
'])

top_depts = dept_counts.head(8).index.tolist()
dept_monthly = (df[df['Dept_Clean'].isin(top_depts)]
                .groupby(['YearMonth', 'Dept_Clean']).size().unstack(fill_value=0))

skill_cols = [c for c in df.columns if c.startswith('skill_')]
skill_totals = df[skill_cols].sum().sort_values(ascending=False)
skill_totals.index = skill_totals.index.str.replace('skill_', '')

df_remote_monthly = (df.groupby('YearMonth')
                    .agg(Total=('Job Name', 'count'),
                        Remote=('Is_Remote', 'sum'),
                        Online=('Is_Online', 'sum'))
                    .reset_index())
df_remote_monthly['Remote_Pct'] = df_remote_monthly['Remote'] /
df_remote_monthly['Total'] * 100
df_remote_monthly['YM_str'] =
df_remote_monthly['YearMonth'].astype(str)

print(" ✓ Metrics computed.")

```

#

3. ML MODELS

#

```

print("\n" + "="*60)
print(" STEP 3: Training ML Models")
print("="*60)

```

— 3a. Random Forest: Classify department

```

le_dept = LabelEncoder()

```

```

le_city = LabelEncoder()
le_exp = LabelEncoder()
le_jt = LabelEncoder()

df2 = df[df['Dept_Clean'] != 'Unknown'].copy()
df2 = df2[df2['Dept_Clean'].isin(dept_counts.head(10).index)]

df2['dept_enc'] = le_dept.fit_transform(df2['Dept_Clean'])
df2['city_enc'] = le_city.fit_transform(df2['City'])
df2['exp_enc'] = le_exp.fit_transform(df2['Exp_Clean'])
df2['jt_enc'] = le_jt.fit_transform(df2['JobType_Clean'])

X_clf = df2[['city_enc', 'exp_enc', 'jt_enc', 'Is_Online', 'Is_Remote'] +
skill_cols]
y_clf = df2['dept_enc']

X_tr, X_te, y_tr, y_te = train_test_split(X_clf, y_clf, test_size=0.2,
random_state=42)
rf = RandomForestClassifier(n_estimators=200, max_depth=12,
random_state=42, n_jobs=-1)
rf.fit(X_tr, y_tr)
rf_score = rf.score(X_te, y_te)
cv_scores = cross_val_score(rf, X_clf, y_clf, cv=5,
scoring='accuracy')
print(f" Random Forest Accuracy : {rf_score*100:.1f}%")
print(f" CV Mean Accuracy : {cv_scores.mean()*100:.1f}% ±
{cv_scores.std()*100:.1f}%")

feat_imp = pd.Series(rf.feature_importances_,
index=X_clf.columns).sort_values(ascending=False).head(15)
feat_imp.index =
feat_imp.index.str.replace('skill_', '').str.replace('_enc', '')

```

— 3b. Time-Series Forecast

```

mc = monthly_counts.copy()
mc['t'] = np.arange(len(mc))
X_ts = mc[['t']].values
y_ts = mc['Count'].values

lr = LinearRegression()
lr.fit(X_ts, y_ts)
mc['Fitted'] = lr.predict(X_ts)

future_t = np.arange(len(mc), len(mc)+6).reshape(-1,1)
future_forecast = lr.predict(future_t)
future_dates = pd.period_range(mc['YearMonth'].iloc[-1]+1,
periods=6, freq='M')

print(f" Forecast (next 6 months): {[int(v) for v in

```

```
future_forecast]})
```

— 3c. K-Means Clustering

```
city_dept = pd.crosstab(df['City'],
df['Dept_Clean'].replace('Unknown', np.nan).dropna())
city_dept = city_dept.loc[city_dept.sum(axis=1) > 20]
scaler = StandardScaler()
X_km = scaler.fit_transform(city_dept)
km = KMeans(n_clusters=3, random_state=42, n_init=10)
city_dept['Cluster'] = km.fit_predict(X_km)
cluster_labels = {0: 'Tech Hub', 1: 'Services Hub', 2: 'Mixed Market'}
city_dept['Cluster_Label'] = city_dept['Cluster'].map(cluster_labels)
print(" ✓ All models trained.")
```

```
#
```

4. VISUALIZATIONS

```
#
```

```
print("\n" + "="*60)
print(" STEP 4: Generating Visualizations")
print("="*60)
```

```
FIG_PATHS = {}
```

— FIG 1: Monthly job postings + trend

```
fig, ax = plt.subplots(figsize=(14, 5))
x_idx = range(len(mc))
ax.bar(x_idx, mc['Count'], color=PALETTE['blue'], alpha=0.75,
zorder=2, label='Monthly Jobs')
ax.plot(x_idx, mc['Fitted'], color=PALETTE['red'], lw=2.5, ls='--',
label='Trend Line', zorder=3)
fut_x = range(len(mc), len(mc)+6)
ax.plot(fut_x, future_forecast, color=PALETTE['green'], lw=2.5,
ls='-', marker='o',
markersize=6, label='6-Month Forecast', zorder=3)
ax.fill_between(fut_x, future_forecast*0.85, future_forecast*1.15,
color=PALETTE['green'], alpha=0.15, label='Forecast
Band (±15%)')
ax.axvline(len(mc)-0.5, color='gray', ls=':', lw=1.5)
ax.text(len(mc), ax.get_ylim()[1]*0.9, 'Forecast ►', color='gray',
fontsize=9)
ax.set_xticks(list(x_idx)[::2])
ax.set_xticklabels(mc['YearMonth_str'].iloc[::2], rotation=45,
ha='right', fontsize=8)
ax.set_xlabel('Month', fontsize=11)
```

```

ax.set_ylabel('Number of Job Postings', fontsize=11)
ax.set_title('Pakistan Job Market – Monthly Postings & 6-Month
Forecast\n(Dec 2019 – Mar 2021)',
            fontsize=14, fontweight='bold')
ax.legend(fontsize=9)
ax.yaxis.set_major_locator(MaxNLocator(integer=True))
ax.grid(axis='y', alpha=0.4)
plt.tight_layout()
FIG_PATHS['monthly_trend'] = save_fig('01_monthly_trend')

```

— FIG 2: Top Departments

```

fig, ax = plt.subplots(figsize=(12, 7))
dept_top = dept_counts.head(12)
colors_dept = [COLORS[i % len(COLORS)] for i in range(len(dept_top))]
bars = ax.barh(dept_top.index[::-1], dept_top.values[::-1],
               color=colors_dept[::-1],
               edgecolor='white', height=0.7)
for bar, val in zip(bars, dept_top.values[::-1]):
    ax.text(bar.get_width()+10, bar.get_y()+bar.get_height()/2,
            f'{val:,}', va='center', fontsize=9, fontweight='bold')
ax.set_xlabel('Number of Job Postings', fontsize=11)
ax.set_title('Top 12 Departments by Job Postings', fontsize=14,
            fontweight='bold')
ax.set_xlim(0, dept_top.max()*1.15)
ax.grid(axis='x', alpha=0.3)
plt.tight_layout()
FIG_PATHS['top_depts'] = save_fig('02_top_departments')

```

— FIG 3: City distribution

```

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 6))
city_top = city_counts.head(8)
wedge_colors = COLORS[:8]
wedges, texts, autotexts = ax1.pie(city_top.values,
                                   labels=city_top.index,
                                   autopct='%1.1f%%',
                                   colors=wedge_colors,
                                   pctdistance=0.82, startangle=140,
                                   wedgeprops=dict(edgecolor='white',
                                   linewidth=1.5))
for at in autotexts: at.set_fontsize(8)
ax1.set_title('Job Distribution by City', fontsize=12,
            fontweight='bold')
ax2.bar(city_top.index, city_top.values, color=wedge_colors,
        edgecolor='white')
ax2.set_ylabel('Job Count', fontsize=11)
ax2.set_title('Job Count per City (Top 8)', fontsize=12,
            fontweight='bold')

```



```

ax2.tick_params(axis='x', rotation=30)
for i, v in enumerate(city_top.values):
    ax2.text(i, v+15, str(v), ha='center', fontsize=8,
fontweight='bold')
ax2.grid(axis='y', alpha=0.3)
plt.tight_layout()
FIG_PATHS['city_dist'] = save_fig('03_city_distribution')

```

— FIG 4: Experience heatmap

```

fig, ax = plt.subplots(figsize=(13, 6))
exp_city = pd.crosstab(df['City'], df['Exp_Clean'])
exp_city = exp_city.loc[city_counts.head(8).index]
exp_order = ['Fresher', '< 1 Year', '1 Year', '2 Years', '3 Years', '4
Years', '5 Years', '6+ Years', 'Other']
exp_city = exp_city.reindex(columns=[c for c in exp_order if c in
exp_city.columns])
sns.heatmap(exp_city, annot=True, fmt='d', cmap='YlOrRd',
linewidths=0.5,
            linecolor='white', ax=ax, annot_kws={'size': 9})
ax.set_title('Experience Requirement Heatmap: City x Experience
Level', fontsize=13, fontweight='bold')
ax.set_xlabel('Experience Required', fontsize=10)
ax.set_ylabel('City', fontsize=10)
plt.tight_layout()
FIG_PATHS['exp_heatmap'] = save_fig('04_experience_heatmap')

```

— FIG 5: Department trend (stacked area)

```

fig, ax = plt.subplots(figsize=(14, 6))
dept_monthly_plot = dept_monthly.copy()
dept_monthly_plot.index = dept_monthly_plot.index.astype(str)
dept_monthly_plot.plot.area(ax=ax, stacked=True, colormap='tab10',
alpha=0.8)
ax.set_title('Department Trends Over Time (Stacked Area)',
fontsize=13, fontweight='bold')
ax.set_xlabel('Month', fontsize=11)
ax.set_ylabel('Job Count', fontsize=11)
ax.tick_params(axis='x', rotation=45)
ax.legend(loc='upper left', fontsize=8, ncol=2)
ax.grid(axis='y', alpha=0.3)
plt.tight_layout()
FIG_PATHS['dept_trend'] = save_fig('05_department_trend')

```

— FIG 6: Tech Skills Demand

```

fig, ax = plt.subplots(figsize=(13, 6))
sk = skill_totals[skill_totals > 0]
sk_colors = [COLORS[i % len(COLORS)] for i in range(len(sk))]
bars = ax.bar(sk.index, sk.values, color=sk_colors, edgecolor='white',

```

```

width=0.7)
for bar, val in zip(bars, sk.values):
    ax.text(bar.get_x()+bar.get_width()/2, bar.get_height()+5,
            str(int(val)), ha='center', va='bottom', fontsize=8,
            fontweight='bold')
ax.set_title('Technology & Skill Demand in Job Descriptions',
            fontsize=13, fontweight='bold')
ax.set_ylabel('Job Postings Mentioning Skill', fontsize=11)
ax.tick_params(axis='x', rotation=45, labelsz=9)
ax.grid(axis='y', alpha=0.3)
plt.tight_layout()
FIG_PATHS['skills'] = save_fig('06_tech_skills')

```

— FIG 7: Job Label Distribution

```

fig, ax = plt.subplots(figsize=(8, 5))
lc_colors = [PALETTE['purple'], PALETTE['orange'], PALETTE['teal'],
            PALETTE['blue'], PALETTE['red']][:len(label_counts)]
ax.pie(label_counts.values, labels=label_counts.index,
        autopct='%1.1f%%', colors=lc_colors, startangle=90,
        pctdistance=0.8,
        wedgeprops=dict(edgecolor='white', linewidth=2))
ax.set_title('Job Listing Type Distribution\n(Premium vs Hot vs
Standard)',
            fontsize=12, fontweight='bold')
plt.tight_layout()
FIG_PATHS['label_dist'] = save_fig('07_label_distribution')

```

— FIG 8: Remote / Online work trend

```

fig, ax1 = plt.subplots(figsize=(13, 5))
ax2 = ax1.twinx()
ym_str = df_remote_monthly['YM_str']
x_pos = range(len(ym_str))
ax1.bar(x_pos, df_remote_monthly['Total'], color=PALETTE['blue'],
        alpha=0.4, label='Total Jobs')
ax1.bar(x_pos, df_remote_monthly['Remote'], color=PALETTE['green'],
        alpha=0.9, label='Remote Jobs')
ax2.plot(x_pos, df_remote_monthly['Remote_Pct'], color=PALETTE['red'],
        lw=2.5,
        marker='o', markersize=5, label='Remote %')
ax1.set_xticks(list(x_pos)[::2])
ax1.set_xticklabels(list(ym_str)[::2], rotation=45, ha='right',
        fontsize=8)
ax1.set_ylabel('Job Count', fontsize=10)
ax2.set_ylabel('Remote %', fontsize=10, color=PALETTE['red'])
ax2.tick_params(axis='y', labelcolor=PALETTE['red'])
ax1.set_title('Remote Work Trend Over Time', fontsize=13,
        fontweight='bold')

```

```

lines1, labels1 = ax1.get_legend_handles_labels()
lines2, labels2 = ax2.get_legend_handles_labels()
ax1.legend(lines1+lines2, labels1+labels2, loc='upper left',
fontsize=9)
ax1.grid(axis='y', alpha=0.3)
plt.tight_layout()
FIG_PATHS['remote_trend'] = save_fig('08_remote_trend')

```

— FIG 9: Feature Importance

```

fig, ax = plt.subplots(figsize=(10, 6))
fi_colors = [COLORS[i % len(COLORS)] for i in range(len(feat_imp))]
ax.barh(feat_imp.index[::-1], feat_imp.values[::-1],
color=fi_colors[::-1], edgecolor='white')
ax.set_title('Random Forest – Top Feature Importances\n(Predicting Job
Department)',
fontsize=13, fontweight='bold')
ax.set_xlabel('Importance Score', fontsize=11)
ax.grid(axis='x', alpha=0.3)
for i, (val, name) in enumerate(zip(feat_imp.values[::-1],
feat_imp.index[::-1])):
    ax.text(val+0.001, i, f'{val:.3f}', va='center', fontsize=8)
plt.tight_layout()
FIG_PATHS['rf_importance'] = save_fig('09_rf_feature_importance')

```

— FIG 10: Day of week

```

fig, ax = plt.subplots(figsize=(9, 5))
dow_colors = [PALETTE['green'] if v == dow_counts.max() else
PALETTE['blue']
for v in dow_counts.values]
ax.bar(dow_counts.index, dow_counts.values, color=dow_colors,
edgecolor='white', width=0.65)
ax.set_title('Job Postings by Day of Week\n(Employer Behavior
Pattern)',
fontsize=12, fontweight='bold')
ax.set_ylabel('Total Job Postings', fontsize=11)
for i, v in enumerate(dow_counts.values):
    ax.text(i, v+10, str(v), ha='center', fontsize=9,
fontweight='bold')
ax.grid(axis='y', alpha=0.3)
plt.tight_layout()
FIG_PATHS['dow'] = save_fig('10_day_of_week')

```

— FIG 11: City Cluster Scatter

```

fig, ax = plt.subplots(figsize=(10, 6))
cluster_colors_map = {0: PALETTE['blue'], 1: PALETTE['orange'], 2:
PALETTE['green']}
pca = PCA(n_components=2)

```

```

X_2d = pca.fit_transform(X_km)
for cl in range(3):
    mask = city_dept['Cluster'] == cl
    ax.scatter(X_2d[mask, 0], X_2d[mask, 1], c=cluster_colors_map[cl],
s=120,
                label=cluster_labels[cl], edgecolors='white',
linewidth=0.8, zorder=3)
for i, city in enumerate(city_dept.index):
    ax.annotate(city, (X_2d[i,0], X_2d[i,1]), fontsize=8,
                xytext=(4,4), textcoords='offset points')
ax.set_title('City Job Market Clustering (PCA + K-Means)',
fontsize=13, fontweight='bold')
ax.set_xlabel('Principal Component 1', fontsize=10)
ax.set_ylabel('Principal Component 2', fontsize=10)
ax.legend(fontsize=10)
ax.grid(alpha=0.3)
plt.tight_layout()
FIG_PATHS['clusters'] = save_fig('11_city_clusters')

```

— FIG 12: Experience donut

```

fig, ax = plt.subplots(figsize=(8, 6))
exp_order_plot = [e for e in ['Fresher', '< 1 Year', '1 Year', '2
Years', '3 Years',
                                '4 Years', '5 Years', '6+ Years', 'Other']]

if e in exp_counts.index]
exp_vals = exp_counts[exp_order_plot]
exp_cols = COLORS[:len(exp_order_plot)]
ax.pie(exp_vals.values, labels=exp_vals.index, autopct='%1.1f%%',
colors=exp_cols,
        startangle=90, pctdistance=0.78,
        wedgeprops=dict(width=0.55, edgecolor='white', linewidth=2))
ax.set_title('Experience Level Requirements (Donut Chart)',
fontsize=12, fontweight='bold')
centre_circle = plt.Circle((0,0), 0.38, fc='white')
ax.add_patch(centre_circle)
ax.text(0, 0, f'{len(df):,}\nJobs', ha='center', va='center',
fontsize=11, fontweight='bold')
plt.tight_layout()
FIG_PATHS['exp_donut'] = save_fig('12_experience_donut')

```

— FIG 13: Skill Correlation Heatmap

```

fig, ax = plt.subplots(figsize=(12, 9))
skill_df = df[skill_cols].copy()
skill_df.columns = skill_df.columns.str.replace('skill_', '')
corr = skill_df[skill_df.sum(axis=1) > 0].corr()
mask = np.triu(np.ones_like(corr, dtype=bool))
sns.heatmap(corr, mask=mask, annot=True, fmt='.2f', cmap='coolwarm',

```

```

center=0,
    linewidths=0.5, ax=ax, annot_kws={'size':7}, vmin=-0.3,
vmax=0.5)
ax.set_title('Technology Skills Co-occurrence Correlation\n(Jobs
Requiring Multiple Skills)',
    fontsize=12, fontweight='bold')
plt.tight_layout()
FIG_PATHS['skill_corr'] = save_fig('13_skill_correlation')

# — FIG 14: Forecast decomposition

fig, axes = plt.subplots(2, 1, figsize=(14, 8))
x_idx = range(len(mc))
fut_x = range(len(mc), len(mc)+6)
ax = axes[0]
all_x = list(x_idx) + list(fut_x)
all_y = list(mc['Fitted']) + list(future_forecast)
ax.bar(x_idx, mc['Count'], color=PALETTE['blue'], alpha=0.6,
label='Actual')
ax.plot(all_x, all_y, color=PALETTE['red'], lw=2, ls='--',
label='Trend + Forecast')
ax.fill_between(fut_x, np.array(future_forecast)*0.85,
np.array(future_forecast)*1.15,
    color=PALETTE['green'], alpha=0.2, label='Confidence
Band')
ax.axvline(len(mc)-0.5, color='gray', ls=':', lw=1.5)
ax.set_xticks(list(x_idx)[::2])
ax.set_xticklabels(mc['YearMonth_str'].iloc[::2], rotation=45,
ha='right', fontsize=8)
ax.set_title('Job Market Forecast (Linear Trend Model)', fontsize=12,
fontweight='bold')
ax.set_ylabel('Job Count')
ax.legend(fontsize=9)
ax.grid(alpha=0.3)
residuals = mc['Count'].values - mc['Fitted'].values
ax2 = axes[1]
ax2.bar(x_idx, residuals,
    color=[PALETTE['green'] if r >= 0 else PALETTE['red'] for r in
residuals], alpha=0.8)
ax2.axhline(0, color='black', lw=1)
ax2.set_xticks(list(x_idx)[::2])
ax2.set_xticklabels(mc['YearMonth_str'].iloc[::2], rotation=45,
ha='right', fontsize=8)
ax2.set_title('Model Residuals (Actual - Fitted)', fontsize=11,
fontweight='bold')
ax2.set_ylabel('Residual')
ax2.grid(alpha=0.3)
plt.tight_layout()
FIG_PATHS['forecast_detail'] = save_fig('14_forecast_detail')

```

```
print(f"\n    ✓ All 14 figures saved to: {FIG_DIR}")
```

#

5. BUILD PDF REPORT

#

```
print("\n" + "="*60)
```

```
print("    STEP 5: Building PDF Report")
```

```
print("="*60)
```

```
PDF_PATH = os.path.join(OUT_DIR, "Pakistan_Job_Market_Report.pdf")
```

```
doc = SimpleDocTemplate(PDF_PATH, pagesize=A4,
                        leftMargin=1.8*cm, rightMargin=1.8*cm,
                        topMargin=1.5*cm, bottomMargin=2*cm)
```

W, H = A4

$$\text{PAGE } W = W - 3.6^* \text{cm}$$

```
styles = getSampleStyleSheet()
```

```
S_TITLE = ParagraphStyle('CustomTitle',
    fontSize=26, fontName='Helvetica-Bold',
    textColor=HexColor('#2C3E50'), alignment=TA_CENTER,
    spaceAfter=6, leading=30)
```

```
S_SUBTITLE = ParagraphStyle('SubTitle',
    fontSize=13, fontName='Helvetica',
    textColor=HexColor('#7F8C8D'), alignment=TA_CENTER, spaceAfter=4)
```

```
S_H1 = ParagraphStyle('H1',
    fontSize=16, fontName='Helvetica-Bold',
    textColor=HexColor('#2C3E50'), spaceBefore=14, spaceAfter=6,
    leading=20)
```

```
S_H2 = ParagraphStyle('H2',
    fontSize=12, fontName='Helvetica-Bold',
    textColor=HexColor('#2980B9'), spaceBefore=10, spaceAfter=4)
```

```
S_BODY = ParagraphStyle('Body',
    fontSize=9.5, fontName='Helvetica',
    textColor=HexColor('#2C3E50'), leading=15,
    alignment=TA_JUSTIFY, spaceAfter=6)
```

```
S_BULLET = ParagraphStyle('Bullet',
    fontSize=9.5, fontName='Helvetica',
    textColor=HexColor('#2C3E50'), leading=15,
    leftIndent=16, spaceAfter=4)
```

```
S_CAPTION = ParagraphStyle('Caption',
    fontSize=8, fontName='Helvetica-Oblique',
    textColor=HexColor('#7F8C8D'), alignment=TA_CENTER, spaceAfter=8)
```

[illegible]

```

        textColor=HexColor('#154360'), backColor=HexColor('#EBF5FB'),
        borderPad=8, leading=15, spaceAfter=8,
        leftIndent=8, rightIndent=8)

def section_divider():
    return HRFlowable(width="100%", thickness=1.5,
        color=HexColor('#2980B9'),
        spaceAfter=6, spaceBefore=4)

def fig_image(path, caption, width_frac=1.0):
    w = PAGE_W * width_frac
    return [RLImage(path, width=w, height=w*0.45), Paragraph(caption,
        S_CAPTION)]

def kpi_table(kpis):
    data = [[Paragraph(f'<b>{v}</b>',
        ParagraphStyle('KV', fontSize=14,
        fontName='Helvetica-Bold',
        textColor=HexColor(c),
        alignment=TA_CENTER)),
        Paragraph(f'{k}',
        ParagraphStyle('KL', fontSize=8,
        fontName='Helvetica',
        textColor=HexColor('#566573'),
        alignment=TA_CENTER))]]
    for k, v, c in kpis]
    rows = []
    for i in range(0, len(data), 4):
        rows.append([data[j][0] for j in range(i, min(i+4,
        len(data)))]])
        rows.append([data[j][1] for j in range(i, min(i+4,
        len(data)))]])
    col_w = PAGE_W / 4
    tbl = Table(rows, colWidths=[col_w]*4)
    tbl.setStyle(TableStyle([
        ('ALIGN', (0,0), (-1,-1), 'CENTER'),
        ('VALIGN', (0,0), (-1,-1), 'MIDDLE'),
        ('ROWBACKGROUNDS', (0,0), (-1,-1), [HexColor('#EBF5FB'),
        HexColor('#FDFEFE')]),
        ('BOX', (0,0), (-1,-1), 0.5, HexColor('#BDC3C7')),
        ('INNERGRID', (0,0), (-1,-1), 0.25, HexColor('#D5D8DC')),
        ('TOPPADDING', (0,0), (-1,-1), 6),
        ('BOTTOMPADDING', (0,0), (-1,-1), 6),
    ]))
    return tbl

# — Build story
story = []

```

```

total_jobs = len(df)
total_cities= df['City'].nunique()
total_depts = df['Dept_Clean'].nunique()
total_cos = df['Company Name'].nunique()
pct_remote = df['Is_Remote'].mean() * 100
peak_month = mc.loc[mc['Count'].idxmax(), 'YearMonth_str']
peak_count = mc['Count'].max()
avg_monthly = mc['Count'].mean()

# Cover page
story += [
    Spacer(1, 1.5*cm),
    Paragraph("Pakistan Job Market", S_TITLE),
    Paragraph("Analysis, Intelligence & Forecasting Report",
S_SUBTITLE),
    Paragraph("December 2019 – March 2021", S_SUBTITLE),
    Spacer(1, 0.4*cm),
    section_divider(),
    Spacer(1, 0.3*cm),
]

kpis = [
    ("Total Job Postings", f"{total_jobs:,}", '#2980B9'),
    ("Cities Covered", str(total_cities), '#27AE60'),
    ("Departments", str(total_depts), '#8E44AD'),
    ("Unique Companies", f"{total_cos:,}", '#E67E22'),
    ("Remote-Friendly Jobs", f"{pct_remote:.1f}%", '#16A085'),
    ("Peak Month Jobs", f"{peak_count:,}", '#E74C3C'),
    ("Avg Monthly Postings", f"{avg_monthly:.0f}", '#2C3E50'),
    ("Analysis Period", "16 Months", '#F39C12'),
]

story.append(kpi_table(kpis))
story.append(Spacer(1, 0.3*cm))
story.append(Paragraph(
    "This report presents a comprehensive data science analysis of the
    Pakistani job market "
    "across 16 months, combining exploratory analysis, statistical
    modeling, machine learning "
    "classification, K-Means clustering, and time-series forecasting
    to surface actionable insights "
    "for job seekers, employers, and policy makers.",
    S_BODY))
story.append(PageBreak())

# Section 1
story += [
    Paragraph("1. Market Overview & Monthly Trends", S_H1),
    section_divider(),
    Paragraph(
        f"The dataset encompasses <b>{total_jobs:,}</b> job postings</b>

```



```

across <b>{total_cities} cities</b> "
    f"from December 2019 to March 2021. Monthly volumes peaked at
<b>{peak_count:,} jobs in {peak_month}</b> "
    f"and averaged <b>{avg_monthly:.0f} postings per month</b>. "
    "Linear regression on the monthly series yields a
statistically meaningful upward trajectory, "
    "suggesting sustained demand recovery despite COVID-19
disruptions.", S_BODY),
]
story += fig_image(FIG_PATHS['monthly_trend'],
    "Figure 1 – Monthly job postings with linear trend and 6-month
forward forecast (shaded confidence band ±15%)")
story.append(Paragraph(
    f"The 6-month forecast projects average monthly postings reaching
~{int(future_forecast[-1]):,} "
    "by end-2021, confirming a healthy and growing market
trajectory.", S_INSIGHT))
story.append(PageBreak())

# Section 2
story += [
    Paragraph("2. Sector & Geographic Distribution", S_H1),
    section_divider(),
    Paragraph(
        f"IT Jobs dominate with <b>{dept_counts.iloc[0]:,} postings
({dept_counts.iloc[0]/total_jobs*100:.1f}%)</b>, "
        "followed by Customer Service and Sales. Karachi and Lahore
account for the majority of listings.", S_BODY),
]
story += fig_image(FIG_PATHS['top_depts'], "Figure 2 – Top 12
departments by total job postings")
story += fig_image(FIG_PATHS['city_dist'], "Figure 3 – Geographic
distribution of job postings")
story.append(PageBreak())

# Section 3
story += [
    Paragraph("3. Experience Requirements & Technical Skills", S_H1),
    section_divider(),
    Paragraph(
        "Fresh graduates and candidates with fewer than 2 years
experience make up the majority of openings. "
        "PHP, SQL, and JavaScript top the skill demand list – followed
by .NET and Python, "
        "signaling growing enterprise and data-science adoption.",
S_BODY),
]
story += fig_image(FIG_PATHS['exp_heatmap'], "Figure 4 – Experience
requirement heatmap: city vs experience level")

```

```
story += fig_image(FIG_PATHS['exp_donut'], "Figure 5 – Overall  
experience requirement distribution")  
story += fig_image(FIG_PATHS['skills'], "Figure 6 – Technology &  
skill demand from job descriptions")  
story.append(Paragraph(  
    "Strategic Insight: Python, Machine Learning, and Data Science  
mentions are rising sharply. "  
    "PHP and WordPress remain the most-needed skills for freelance and  
SME-focused markets.", S_INSIGHT))  
story.append(PageBreak())
```

Section 4

```
story += [  
    Paragraph("4. Temporal & Behavioral Patterns", S_H1),  
    section_divider(),  
    Paragraph(  
        "IT and Customer Service are persistently dominant. Employers  
post predominantly on weekdays, "  
        "with Monday and Tuesday the busiest days – a pattern useful  
for job seekers to time their applications.", S_BODY),  
    ]  
story += fig_image(FIG_PATHS['dept_trend'], "Figure 7 – Department-  
level posting trends (stacked area)")  
story += fig_image(FIG_PATHS['dow'], "Figure 8 – Day-of-week  
posting behavior")  
story.append(PageBreak())
```

Section 5

```
story += [  
    Paragraph("5. Remote & Online Work Trends", S_H1),  
    section_divider(),  
    Paragraph(  
        "Remote-friendly job postings have grown markedly since Q1  
2020, a direct consequence of COVID-19 "  
        "accelerating digital-work adoption. The trend is unmistakably  
upward.", S_BODY),  
    ]  
story += fig_image(FIG_PATHS['remote_trend'], "Figure 9 – Remote work  
volume and percentage trend")  
story.append(Paragraph(  
    "Future Outlook: Remote and hybrid job postings are expected to  
grow 15-20% year-on-year "  
    "as employers formalise flexible working arrangements.",  
S_INSIGHT))  
story.append(PageBreak())
```

Section 6

```
story += [  
    Paragraph("6. Machine Learning & Predictive Analytics", S_H1),  
    section_divider(),
```

```

Paragraph("<b>6.1 Department Classification (Random Forest)</b>",
S_H2),
Paragraph(
    f"A Random Forest classifier achieved <b>{rf_score*100:.1f}%
test accuracy</b> with a "
    f"5-fold CV mean of <b>{cv_scores.mean()*100:.1f}% (±
{cv_scores.std()*100:.1f}%)</b>."
    "City location and key skills such as PHP and SQL were the
strongest predictors.", S_BODY),
]
story += fig_image(FIG_PATHS['rf_importance'], "Figure 10 – Random
Forest top-15 feature importances")
story += [Paragraph("<b>6.2 Time-Series Forecasting (Linear
Regression)</b>", S_H2)]
story += fig_image(FIG_PATHS['forecast_detail'], "Figure 11 – Forecast
with residual analysis")
story += [Paragraph("<b>6.3 City Market Clustering (K-Means)</b>",
S_H2),
Paragraph(
    "K-Means (k=3) identifies: Tech Hubs (Islamabad, Lahore), "
    "Services Hubs (Karachi), and Mixed Markets (Faisalabad,
Multan).", S_BODY)]
story += fig_image(FIG_PATHS['clusters'], "Figure 12 – City job market
clusters (K-Means + PCA)")
story.append(PageBreak())

# Section 7
story += [
    Paragraph("7. Hidden Insights & Skill Co-occurrence", S_H1),
    section_divider(),
    Paragraph(
        "Analysing skill co-occurrence reveals natural skill bundles –
groups of technologies "
        "employers consistently list together, defining effective
learning pathways.", S_BODY),
]
story += fig_image(FIG_PATHS['skill_corr'], "Figure 13 – Technology
skill co-occurrence correlation matrix")
story.append(Paragraph(
    "PHP & WordPress and Angular & Node co-occur strongly. Python,
Machine Learning, and Data Science "
    "form a tightly coupled demand cluster – integrated upskilling is
more valuable than single-skill certifications.",
    S_INSIGHT))
story += fig_image(FIG_PATHS['label_dist'], "Figure 14 – Job listing
type distribution")
story.append(PageBreak())

# Section 8

```

```

story += [
    Paragraph("8. Strategic Insights & Future Trends", S_H1),
    section_divider(),
    Paragraph("<b>For Job Seekers</b>", S_H2),
]
for bullet in [
    "IT is king – over 30% of all postings are IT-related; Python, JavaScript, and PHP skills command the widest demand.",
    "Apply on Monday or Tuesday – employers post most heavily early in the week.",
    "Remote work is rising – targeting remote roles opens access to international salaries.",
    "Fresh graduates are in demand – 40%+ of postings welcome 0-2 years experience.",
    "Bundle skills – pairing PHP+WordPress or Angular+NodeJS significantly boosts hirability.",
]:
    story.append(Paragraph(f"• {bullet}", S_BULLET))

story += [Spacer(1, 0.3*cm), Paragraph("<b>For Employers & Recruiters</b>", S_H2)]
for bullet in [
    "Lahore is underserved – fewer postings per capita than Karachi; talent pool arbitrage opportunity.",
    "AI/ML skills scarcity – Data Science mentions are rising faster than supply.",
    "Post mid-week – Wednesday/Thursday postings face less competition for candidate attention.",
    "Formalise remote hiring – attracts a wider talent pool with minimal overhead cost.",
]:
    story.append(Paragraph(f"• {bullet}", S_BULLET))

story += [
    Spacer(1, 0.3*cm),
    Paragraph("<b>Market Forecast (Apr – Sep 2021)</b>", S_H2),
    Paragraph(
        f"Monthly job postings are projected to grow from ~{int(future_forecast[0]):,} in April 2021 "
        f"to ~{int(future_forecast[-1]):,} by September 2021 – an estimated "
        f"<b>{((future_forecast[-1]/avg_monthly)-1)*100:.1f}% increase</b> over the historical average. "
        "IT, Customer Service, and Computer Software will continue to lead.", S_BODY),
    PageBreak(),
    Paragraph("9. Methodology & Data Quality", S_H1),
    section_divider(),
    Paragraph(

```

```

        f"<b>Dataset:</b> {total_jobs:,} job postings, December 2019 –
March 2021. "
        "<b>Cleaning:</b> date parsing, column whitespace removal,
experience label normalisation, "
        "department suffix standardisation. "
        "<b>Feature engineering:</b> binary remote/online flags, 22
binary skill columns from JD keyword matching, "
        "temporal features. "
        "<b>Models:</b> Random Forest (200 trees, depth 12), 5-fold
CV; "
        "Linear Regression on monthly time index; K-Means (k=3, 10
inits). "
        "<b>Visualisation:</b> Matplotlib + Seaborn. <b>Report:</b>
ReportLab.", S_BODY),
        Spacer(1, 0.5*cm),
        HRFlowable(width="100%", thickness=0.5,
color=HexColor('#BDC3C7')),
        Paragraph(
            f"Pakistan Job Market Report | Generated:
{datetime.now().strftime('%B %d, %Y')} | "
            "Data Science Project – Pakistan Job Market Dataset (Dec 2019
– Mar 2021)",
            ParagraphStyle('Footer', fontSize=7.5,
textColor=HexColor('#AEB6BF'),
                        alignment=TA_CENTER, spaceBefore=4)),
        ]

doc.build(story)

print(f"\n{'='*60}")
print(f"  ☐  ALL DONE!")
print(f"{'='*60}")
print(f"  ☐ Output folder : {OUT_DIR}")
print(f"  ☐ Figures folder: {FIG_DIR}")
print(f"  ☐ PDF Report    : {PDF_PATH}")
print(f"{'='*60}\n")

```

STEP 1: Loading & Cleaning Data

```

Raw shape      : (6680, 9)
Columns       : ['Job Name', 'label', 'Company Name', 'Job Type',
'Experience Required', 'Department', 'JD', 'City', 'Date Posted']
Missing values:
Job Name      0
label         5565
Company Name  663
Job Type      0
Experience Required  0

```

```
Department      0
JD               0
City             0
Date Posted     0
dtype: int64
```

```
Cleaned shape : (6680, 41)
✓ Data cleaning complete.
```

```
=====
STEP 2: Computing Metrics
=====
```

```
✓ Metrics computed.
```

```
=====
STEP 3: Training ML Models
=====
```

```
Random Forest Accuracy : 33.9%
CV Mean Accuracy       : 34.8% ± 5.1%
Forecast (next 6 months): [701, 734, 768, 801, 835, 868]
✓ All models trained.
```

```
=====
STEP 4: Generating Visualizations
=====
```

```
✓ Saved figure: 01_monthly_trend.png
✓ Saved figure: 02_top_departments.png
✓ Saved figure: 03_city_distribution.png
✓ Saved figure: 04_experience_heatmap.png
✓ Saved figure: 05_department_trend.png
✓ Saved figure: 06_tech_skills.png
✓ Saved figure: 07_label_distribution.png
✓ Saved figure: 08_remote_trend.png
✓ Saved figure: 09_rf_feature_importance.png
✓ Saved figure: 10_day_of_week.png
✓ Saved figure: 11_city_clusters.png
✓ Saved figure: 12_experience_donut.png
✓ Saved figure: 13_skill_correlation.png
✓ Saved figure: 14_forecast_detail.png

✓ All 14 figures saved to: C:\Users\asus\JobMarketAnalysis\figures
```

```
=====
STEP 5: Building PDF Report
=====
```

```
☐ ALL DONE!
```

```
☐ Output folder : C:\Users\asus\JobMarketAnalysis
```

□ Figures folder: C:\Users\asus\JobMarketAnalysis\figures
□ PDF Report : C:\Users\asus\JobMarketAnalysis\
Pakistan_Job_Market_Report.pdf

pip install reportlab

Collecting reportlab

Downloading reportlab-4.4.10-py3-none-any.whl.metadata (1.7 kB)

Requirement already satisfied: pillow>=9.0.0 in c:\users\asus\
anaconda3\lib\site-packages (from reportlab) (11.1.0)

Requirement already satisfied: charset-normalizer in c:\users\asus\
anaconda3\lib\site-packages (from reportlab) (3.3.2)

Downloading reportlab-4.4.10-py3-none-any.whl (2.0 MB)

-----	0.0/2.0 MB	?	eta	-:--:--
-----	0.0/2.0 MB	?	eta	-:--:--
-----	0.0/2.0 MB	?	eta	-:--:--
-----	0.0/2.0 MB	?	eta	-:--:--
-----	0.0/2.0 MB	?	eta	-:--:--
-----	0.3/2.0 MB	?	eta	-:--:--
-----	0.3/2.0 MB	?	eta	-:--:--
-----	0.3/2.0 MB	?	eta	-:--:--
-----	0.3/2.0 MB	?	eta	-:--:--
0:00:05	0.5/2.0 MB	311.1 kB/s	eta	
-----	0.5/2.0 MB	311.1 kB/s	eta	
0:00:05	0.5/2.0 MB	311.1 kB/s	eta	
-----	0.5/2.0 MB	311.1 kB/s	eta	
0:00:05	0.5/2.0 MB	311.1 kB/s	eta	
-----	0.8/2.0 MB	292.5 kB/s	eta	
0:00:04	0.8/2.0 MB	292.5 kB/s	eta	
-----	0.8/2.0 MB	292.5 kB/s	eta	
0:00:04	0.8/2.0 MB	292.5 kB/s	eta	
-----	0.8/2.0 MB	292.5 kB/s	eta	
0:00:04	0.8/2.0 MB	292.5 kB/s	eta	
-----	1.0/2.0 MB	299.5 kB/s	eta	
0:00:04	1.0/2.0 MB	299.5 kB/s	eta	
-----	1.0/2.0 MB	299.5 kB/s	eta	
0:00:04	1.0/2.0 MB	299.5 kB/s	eta	
-----	1.0/2.0 MB	299.5 kB/s	eta	
0:00:04	1.0/2.0 MB	299.5 kB/s	eta	
-----	1.0/2.0 MB	299.5 kB/s	eta	
0:00:04	1.0/2.0 MB	299.5 kB/s	eta	

```
----- 1.3/2.0 MB 282.6 kB/s eta
0:00:03
----- 1.3/2.0 MB 282.6 kB/s eta
0:00:03
----- 1.3/2.0 MB 282.6 kB/s eta
0:00:03
----- 1.3/2.0 MB 282.6 kB/s eta
0:00:03
----- 1.3/2.0 MB 282.6 kB/s eta
0:00:03
----- 1.3/2.0 MB 282.6 kB/s eta
0:00:03
----- 1.6/2.0 MB 264.1 kB/s eta
0:00:02
----- 1.6/2.0 MB 264.1 kB/s eta
0:00:02
----- 1.6/2.0 MB 264.1 kB/s eta
0:00:02
----- 1.6/2.0 MB 264.1 kB/s eta
0:00:02
----- 1.6/2.0 MB 264.1 kB/s eta
0:00:02
----- 1.8/2.0 MB 262.6 kB/s eta
0:00:01
----- 1.8/2.0 MB 262.6 kB/s eta
0:00:01
----- 2.0/2.0 MB 263.5 kB/s eta
0:00:00
Installing collected packages: reportlab
Successfully installed reportlab-4.4.10
Note: you may need to restart the kernel to use updated packages.
```